

Git y GitHub: guía diaria para profesionales

Referencia práctica en español · Git (línea de comandos) + GitHub CLI (gh) · Agosto de 2026

Objetivo: cubrir el flujo de trabajo más habitual: configurar identidad, crear ramas, guardar cambios, sincronizar, resolver conflictos, revisar historial, colaborar mediante pull requests y automatizar tareas comunes. Sustituye los valores entre `<...>` por los tuyos.

1. Preparación inicial

Comando	Uso profesional
<code>git --version</code>	Comprueba que Git está instalado.
<code>git config --global user.name "Tu Nombre"</code>	Define el nombre que aparecerá en tus commits.
<code>git config --global user.email "tu@correo.com"</code>	Define el correo asociado a tus commits.
<code>git config --global init.defaultBranch main</code>	Usa <code>main</code> como rama inicial para repositorios nuevos.
<code>git config --global pull.rebase true</code>	Hace que <code>git pull</code> use rebase por defecto; útil para un historial lineal si el equipo lo adopta.
<code>git config --global core.autocrlf input</code>	En macOS/Linux, evita convertir finales de línea al confirmar. En Windows suele usarse <code>true</code> .
<code>git config --global --list</code>	Revisa la configuración global efectiva.
<code>gh auth login</code>	Inicia sesión en GitHub desde la terminal y configura autenticación.
<code>gh auth status</code>	Comprueba la cuenta y el método de autenticación activos.

2. Crear, clonar y explorar

Comando	Uso profesional
<code>git init</code>	Inicializa un repositorio Git en el directorio actual.
<code>git clone https://github.com/<organizacion>/<repo>.git</code>	Clona un repositorio existente mediante HTTPS.
<code>git clone git@github.com:<organizacion>/<repo>.git</code>	Clona mediante SSH; evita pedir credenciales repetidamente.
<code>git status</code>	Muestra rama actual, archivos modificados, preparados y sin seguimiento.
<code>git remote -v</code>	Lista remotos configurados y sus URL.
<code>git remote add origin <url></code>	Añade el remoto principal si el repositorio aún no lo tiene.
<code>git fetch --all --prune</code>	Descarga referencias actualizadas de todos los remotos y elimina referencias remotas obsoletas.
<code>git branch -a</code>	Lista ramas locales y remotas.

Comando	Uso profesional
<code>git log --oneline --graph --decorate --all</code>	Vista compacta y gráfica del historial de todas las ramas.

3. Cambios y commits

Comando	Uso profesional
<code>git diff</code>	Inspecciona cambios aún no preparados.
<code>git diff --staged</code>	Inspecciona lo que entrará en el próximo commit.
<code>git add <archivo></code>	Prepara un archivo concreto.
<code>git add -p</code>	Prepara cambios por fragmentos; excelente para commits pequeños y coherentes.
<code>git restore <archivo></code>	Descarta cambios sin preparar de un archivo. Úsalo con cautela.
<code>git restore --staged <archivo></code>	Saca un archivo del área de preparación, sin descartar sus cambios.
<code>git commit -m "feat: añade validación de correo"</code>	Crea un commit. Usa mensajes claros y consistentes, por ejemplo Conventional Commits.
<code>git commit --amend</code>	Modifica el último commit local; no lo uses en commits ya compartidos sin acordarlo.
<code>git show --stat</code>	Muestra el último commit y un resumen de los archivos afectados.
<code>git blame <archivo></code>	Indica qué commit y autor modificaron cada línea actual.

Patrón de mensaje recomendado

tipo(ámbito opcional): descripción breve

Tipos frecuentes: feat, fix, docs, refactor, test, chore. Ejemplo: `fix(api): maneja expiración del token`.

4. Ramas e integración

Comando	Uso profesional
<code>git switch -c feature/<nombre></code>	Crea y cambia a una nueva rama de trabajo.
<code>git switch <rama></code>	Cambia de rama.
<code>git branch -vv</code>	Muestra ramas, su último commit y la rama remota asociada.
<code>git branch -d <rama></code>	Borra una rama local que ya fue integrada.
<code>git branch -D <rama></code>	Fuerza el borrado local; úsalo solo si entiendes qué trabajo se perderá.
<code>git merge <rama></code>	Integra una rama en la rama actual.
<code>git rebase origin/main</code>	Reaplica tus commits sobre la versión más reciente de main. Hazlo antes de abrir o actualizar una PR, si la política del equipo lo permite.
<code>git rebase --continue</code>	Continúa un rebase después de resolver conflictos y preparar los archivos.
<code>git rebase --abort</code>	Cancela un rebase y vuelve al estado anterior.
<code>git cherry-pick <sha></code>	Aplica un commit específico de otra rama; útil para trasladar un arreglo puntual.

5. Sincronización segura

Comando	Uso profesional
<code>git pull --rebase origin main</code>	Actualiza tu rama local con main evitando un commit de merge automático.
<code>git push -u origin feature/<nombre></code>	Publica una rama por primera vez y configura su seguimiento remoto.
<code>git push</code>	Publica commits de una rama que ya tiene upstream.
<code>git push --force-with-lease</code>	Actualiza de forma más segura una rama propia tras rebase; se niega si el remoto cambió inesperadamente. No usar en ramas compartidas.
<code>git fetch origin</code>	Descarga cambios del remoto sin modificar tu rama de trabajo.
<code>git log HEAD..origin/main --oneline</code>	Muestra commits que están en el remoto y no en tu rama actual.
<code>git log origin/main..HEAD --oneline</code>	Muestra tus commits locales que todavía no están en origin/main.

6. Conflictos y recuperación

Comando	Uso profesional
<code>git status</code>	Primer paso ante un conflicto: enumera archivos afectados y próximos pasos.
<code>git add <archivo-resuelto></code>	Marca como resuelto un archivo tras editar los marcadores de conflicto.
<code>git merge --continue</code>	Finaliza una fusión después de resolver conflictos.
<code>git merge --abort</code>	Cancela una fusión en curso.
<code>git revert <sha></code>	Crea un nuevo commit que revierte uno anterior; opción segura para historial compartido.
<code>git reset --soft HEAD~1</code>	Deshace el último commit conservando los cambios preparados.
<code>git reset --mixed HEAD~1</code>	Deshace el último commit conservando cambios, pero fuera del área de preparación.
<code>git reset --hard HEAD~1</code>	Descarta el último commit y sus cambios locales. Riesgoso; verifica antes.
<code>git reflog</code>	Muestra movimientos recientes de HEAD; clave para recuperar commits o ramas aparentemente perdidos.
<code>git restore --source=<sha> -- <archivo></code>	Restaura la versión de un archivo desde un commit específico.

7. Stash, etiquetas y limpieza

Comando	Uso profesional
<code>git stash push -m "wip: descripción"</code>	Guarda temporalmente cambios sin confirmar.
<code>git stash list</code>	Lista los stashes disponibles.
<code>git stash pop</code>	Restaura y elimina el último stash; puede generar conflictos.
<code>git stash apply stash@{0}</code>	Restaura un stash sin eliminarlo.
<code>git tag -a v1.2.0 -m "Release v1.2.0"</code>	Crea una etiqueta anotada para una versión.

Comando	Uso profesional
<code>git push origin v1.2.0</code>	Publica una etiqueta concreta.
<code>git push origin --tags</code>	Publica todas las etiquetas locales aún no enviadas.
<code>git clean -nd</code>	Simula qué archivos no rastreados se eliminarían.
<code>git clean -fd</code>	Elimina archivos y directorios no rastreados. Ejecuta primero <code>-nd</code> .

8. GitHub CLI y pull requests

Comando	Uso profesional
<code>gh repo clone <organizacion>/<repo></code>	Clona un repositorio usando GitHub CLI.
<code>gh repo create <repo> --private --source=. --push</code>	Crea un repositorio privado desde el directorio actual y publica la rama.
<code>gh pr create --base main --head feature/<nombre> --fill</code>	Abre una pull request con título/cuerpo generados desde commits.
<code>gh pr list</code>	Lista pull requests abiertas.
<code>gh pr view --web</code>	Abre en el navegador la PR relacionada con la rama actual.
<code>gh pr checkout <numero></code>	Cambia localmente a la rama de una pull request.
<code>gh pr diff <numero></code>	Muestra el diff de una PR en la terminal.
<code>gh pr review <numero> --approve</code>	Aprueba una PR, si las reglas del repositorio y tu rol lo permiten.
<code>gh pr review <numero> --comment --body "..."</code>	Deja una revisión con comentario general.
<code>gh pr merge <numero> --squash --delete-branch</code>	Integra una PR mediante squash y elimina la rama, si está permitido.
<code>gh issue create --title "..." --body "..."</code>	Crea un issue desde la terminal.
<code>gh issue list --assignee @me</code>	Lista issues asignados a tu cuenta.
<code>gh run list</code>	Lista ejecuciones recientes de GitHub Actions.
<code>gh run view <id> --log-failed</code>	Muestra los logs de los pasos fallidos de una ejecución.

9. Flujo diario sugerido

Un flujo simple para una tarea de producto o corrección:

Comando	Uso profesional
<code>git switch main && git pull --rebase origin main</code>	Parte desde una base actualizada.
<code>git switch -c feature/descripcion-corta</code>	Aísla el cambio en una rama descriptiva.
<code>git add -p && git commit -m "feat: ..."</code>	Crea commits pequeños y revisables mientras trabajas.

Comando	Uso profesional
<code>git fetch origin && git rebase origin/main</code>	Actualiza tu rama antes de compartirla; resuelve y prueba cualquier conflicto.
<code>git push -u origin feature/descripcion-corta</code>	Publica la rama.
<code>gh pr create --base main --fill</code>	Abre la pull request y completa contexto, pruebas y riesgos.
<code>gh pr view --web</code>	Revisa checks, comentarios y estado de aprobación hasta integrar.

10. Buenas prácticas y alertas

Antes de acciones destructivas: usa `git status`, revisa el diff y, cuando sea posible, prueba con variantes de simulación como `git clean -nd`. Prefiere `git revert` en ramas compartidas: preserva el historial y facilita auditoría.

No subas secretos: no confirmes tokens, claves privadas, archivos `.env` ni credenciales. Añádelos a `.gitignore`, rota cualquier secreto expuesto y usa los mecanismos de secretos de GitHub/tu plataforma de despliegue.

Evita reescribir historia ajena: no uses `push --force` en main, ramas protegidas o ramas de otras personas. Si debes reescribir una rama propia publicada, usa `--force-with-lease`.

Una PR útil: mantén el alcance reducido, enlaza el issue, explica el motivo del cambio, indica cómo se probó y responde a cada comentario. Ejecuta pruebas y formateadores antes de pedir revisión.

11. Alias opcionales

Comando	Uso profesional
<code>git config --global alias.st "status -sb"</code>	Estado compacto con la rama y su seguimiento.
<code>git config --global alias.lg "log --oneline --graph --decorate --all"</code>	Historial gráfico compacto.
<code>git config --global alias.unstage "restore --staged"</code>	Atajo para retirar archivos del área de preparación.
<code>git config --global alias.last "log -1 HEAD"</code>	Muestra el último commit.

Nota: cada organización puede imponer convenciones distintas para ramas, firmas de commits, rebase/merge y protección de ramas. Sigue siempre las reglas del repositorio y de tu equipo.